

**SISTEMA DE CATÁLOGO INTELIGENTE PARA LIBRERÍAS: ORGANIZACIÓN JERÁRQUICA
Y BÚSQUEDA EFICIENTE MEDIANTE ESTRUCTURAS DE DATOS NO LINEALES**

Carnet 202502375 – Erick Adrian Patzan Perez

Resumen

Las librerías que administran grandes volúmenes de libros enfrentan dificultades para localizar títulos y para identificar qué obras pertenecen a una misma temática cuando el catálogo crece más allá de unos pocos miles de registros. Este ensayo describe el diseño e implementación de un sistema de catálogo bibliográfico desarrollado en C#, orientado a organizar los libros de una librería mediante una estructura jerárquica de categorías y subcategorías, similar a la disposición física de una biblioteca. La solución combina dos tipos de dato abstracto construidos desde cero: un árbol N-ario para representar la jerarquía de categorías, y una tabla hash con manejo de colisiones mediante encadenamiento para realizar búsquedas por ISBN en tiempo prácticamente constante. El sistema permite cargar información de forma incremental mediante archivos XML, generar reportes visuales del catálogo con Graphviz, y se expone al usuario mediante una interfaz web en ASP.NET Core Razor Pages. Los resultados muestran tiempos de búsqueda reducidos sin recurrir a colecciones genéricas del lenguaje.

Palabras clave

Árbol N-ario; tabla hash; tipo de dato abstracto; Graphviz; catálogo bibliográfico.

Abstract

Bookstores that manage large volumes of books face difficulties locating titles and identifying which works belong to the same subject area once the catalog grows beyond a few thousand records. This essay describes the design and implementation of a bibliographic catalog system developed in C#, aimed at organizing a bookstore's books through a hierarchical structure of categories and subcategories, similar to the physical layout of a library. The solution combines two abstract data types built from scratch: an N-ary tree to represent the category hierarchy, and a hash table with collision handling through chaining to search by ISBN in nearly constant time. The system allows information to be loaded incrementally through XML files, generates visual reports using Graphviz, and is exposed to the user through a web interface built with ASP.NET Core Razor Pages. Results show reduced search times without relying on the language's generic collections.

Keywords

N-ary tree; hash table; abstract data type; Graphviz; bibliographic catalog.

Introducción

El manejo eficiente de grandes volúmenes de información es uno de los retos centrales de la ingeniería de software, particularmente cuando los datos poseen una relación jerárquica natural, como ocurre con las categorías de una librería. El presente ensayo expone el diseño de un sistema de catálogo para una cadena de librerías que administra más de cien mil libros distribuidos en distintas sucursales, con el propósito de reducir el tiempo que el personal invierte en localizar títulos y en identificar libros relacionados por temática. Se plantea como base teórica el uso de tipos de dato abstracto no lineales, concretamente árboles y tablas hash, implementados enteramente por el autor sin recurrir a las estructuras genéricas provistas por C#. Se busca responder a la interrogante de cómo lograr búsquedas eficientes sin sacrificar la organización jerárquica que exige la operación diaria de la librería.

Desarrollo del tema

El desarrollo de la solución se organizó en cinco subtemas: el planteamiento del problema, el diseño de las estructuras de datos, la arquitectura orientada a objetos, la manipulación de archivos XML, y la generación de reportes gráficos.

a. Planteamiento del problema y objetivos

La gerencia de la librería requiere un sistema que funcione como una biblioteca física organizada por secciones y estanterías: un mapa general del cual se derivan áreas principales, categorías más específicas y, finalmente, los libros individuales. A partir de esta descripción se identificaron dos necesidades técnicas centrales: (1) representar relaciones de jerarquía y clasificación entre categorías y subcategorías, y (2) localizar libros de forma casi inmediata a partir de su código ISBN, incluso cuando el catálogo alcance decenas de miles de registros.

b. Diseño de las estructuras de datos: árbol de categorías y tabla hash

Para representar la jerarquía de categorías se implementó un árbol N-ario mediante la técnica conocida como "primer hijo, siguiente hermano": cada nodo (NodoCategoria) mantiene una referencia a su primera subcategoría y otra a la siguiente categoría del mismo nivel, lo que permite que cualquier categoría tenga un número arbitrario de subcategorías sin depender de colecciones genéricas. Las subcategorías se insertan manteniendo orden alfabético entre hermanos, cumpliendo el requisito de presentación ordenada del catálogo.

Para la búsqueda de libros se diseñó una tabla hash propia. La función utilizada para determinar la posición de un libro dentro de la tabla se muestra en la ecuación (1):

$$h(k) = k \bmod m \quad (1)$$

donde k es el ISBN del libro y m es el tamaño fijo de la tabla (un número primo, para distribuir mejor las claves). Como distintos ISBN pueden producir la misma posición (colisión), cada posición de la tabla —llamada casilla— almacena en realidad una lista enlazada de libros, resuelta con nodos propios (NodoCasilla y NodoLibro) en lugar de arreglos o colecciones nativas de C#. La Tabla I resume la mejora de complejidad obtenida frente a una búsqueda secuencial sobre una lista enlazada simple.

Operación	Lista enlazada (secuencial)	Tabla hash propia
Búsqueda por ISBN	$O(n)$	$O(1)$ en promedio
Inserción	$O(1)$	$O(1)$ en promedio
Eliminación	$O(n)$	$O(1)$ en promedio dentro de su casilla

Tabla I.

Comparación de complejidad temporal entre búsqueda secuencial y la tabla hash implementada.

Fuente: elaboración propia.

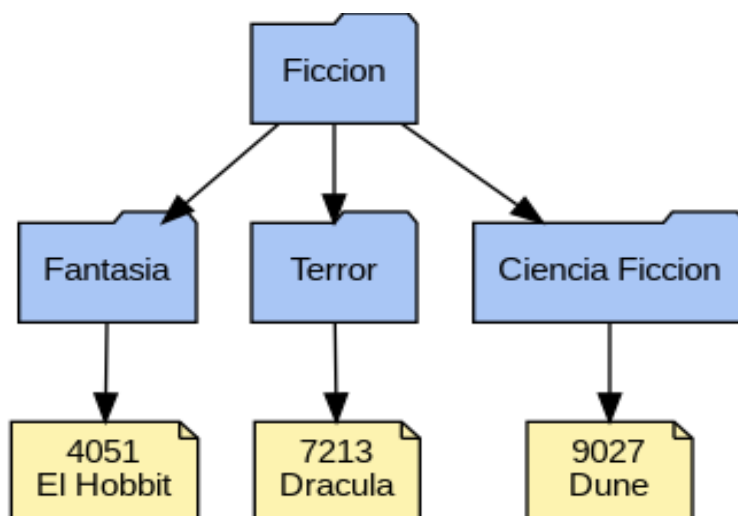


Figura 1. Ejemplo del árbol de categorías, con libros como nodos hoja.

Fuente: elaboración propia.

c. Arquitectura orientada a objetos de la solución

El proyecto se dividió en tres componentes: una biblioteca de estructuras de datos (Proyecto2.TDA), que contiene únicamente las clases Libro, NodoLibro, NodoCasilla, TablaHash, NodoCategoria y ArbolCategorias; una capa de lógica de negocio (Proyecto2.Core), donde la clase Catalogo compone internamente al árbol y a la tabla hash y expone operaciones de alto nivel como RegistrarLibro, BuscarPorIsbn y MostrarEstructura; y el proyecto web (IPC2_Proj2), construido con ASP.NET Core Razor Pages, cuyas páginas únicamente invocan los métodos de Catalogo sin conocer los detalles internos del árbol ni de la tabla hash. Esta separación por capas favorece el principio de encapsulamiento del paradigma orientado a objetos.

d. Manipulación de archivos XML

La carga inicial y las cargas incrementales del catálogo se realizan a partir de archivos XML con la estructura definida en el enunciado del proyecto (listaCategorias y listaLibros, ambos opcionales). La clase LectorXml utiliza la API de System.Xml.Linq para recorrer el documento y, dado que una subcategoría puede aparecer en el archivo antes que su categoría padre, aplica un algoritmo de múltiples pasadas: en cada pasada intenta agregar las categorías pendientes, hasta que una pasada completa no logre insertar ninguna adicional.

e. Generación de reportes gráficos con Graphviz

Para visualizar la organización del catálogo se implementó la clase GeneradorGraphviz, que recorre recursivamente una categoría y todas sus subcategorías, construyendo un archivo de texto en formato DOT: cada categoría se representa como un nodo, cada libro como un nodo hoja, y las relaciones padre-hijo como aristas dirigidas. El archivo .dot resultante se procesa invocando al ejecutable externo dot (parte de Graphviz), que genera una imagen en formato PNG mostrada posteriormente en la interfaz web.

Conclusiones

El uso de tipos de dato abstracto implementados desde cero, en lugar de las colecciones genéricas de C#, permitió comprender a profundidad el funcionamiento interno de un árbol N-ario y de una tabla hash con manejo de colisiones, en vez de tratarlos como cajas negras provistas por el lenguaje.

La combinación de un árbol de categorías con una tabla hash resultó adecuada para el problema planteado: el árbol resuelve la necesidad de organización jerárquica y de visualización de la estructura del catálogo, mientras que la tabla hash resuelve la necesidad de búsquedas rápidas por ISBN, incluso si el catálogo creciera a los cien mil libros mencionados en el planteamiento del problema.

Como trabajo futuro, se recomienda incorporar un mecanismo de redimensionamiento dinámico de la tabla hash cuando el factor de carga supere un umbral determinado, así como persistencia en disco del catálogo entre ejecuciones del sistema.

Referencias bibliográficas

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Ellson, J., Gansner, E., Koutsofios, L., North, S. C., & Woodhull, G. (2004). Graphviz and dynagraph: Static and dynamic graph drawing tools. En Graph Drawing Software (pp. 127-148). Springer.

Freeman, A. (2022). Pro ASP.NET Core (10.a ed.). Apress.

Microsoft. (2024). ASP.NET Core documentation. Microsoft Learn. <https://learn.microsoft.com/aspnet/core>

Weiss, M. A. (2020). Data structures and algorithm analysis in C++ (4th ed.). Pearson.

Apéndice A. Diagrama de clases

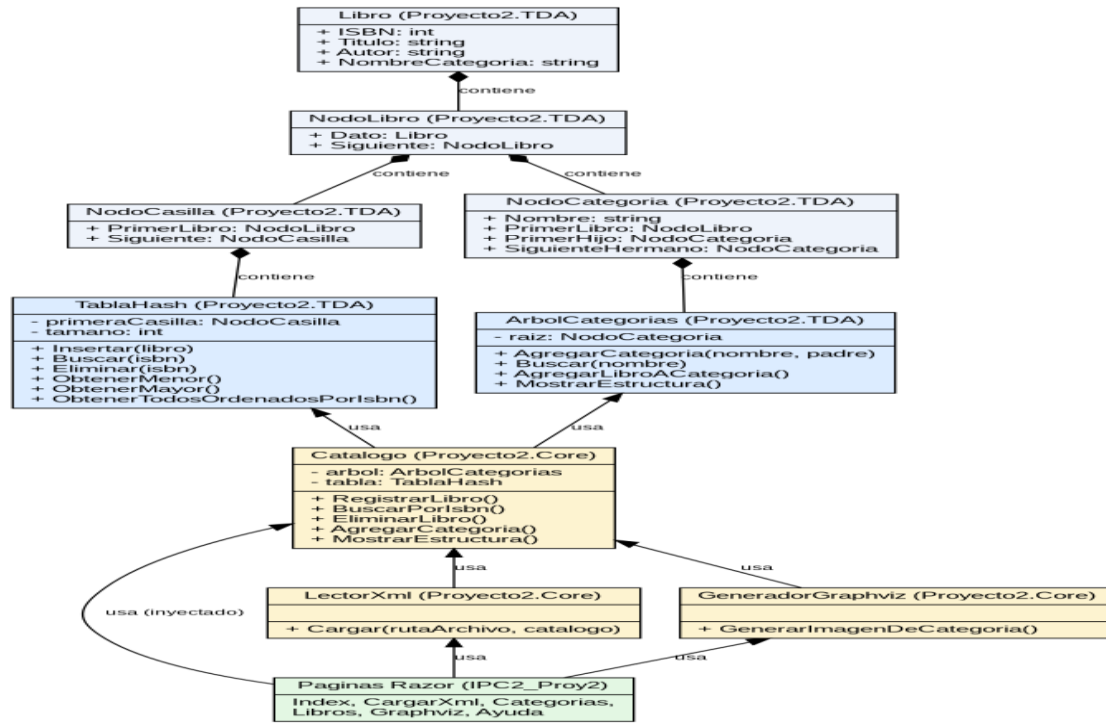


Figura 2. Diagrama de clases del sistema de catálogo.

Fuente: elaboración propia.